

산학 팀 14 BestFriend

Friendly

: 플레이어의 반응에 따라 동적인 공포 이벤트를 제공하는
AI 심리 공포게임

2026-1 이화여자대학교 캡스톤디자인과창업프로젝트B
2차 보고서

산학 팀 14 BestFriend

2271037 양동선

2271030 박주영

2376179 윤소진

지도교수 민동보

목차

- 4. Team Information
 - 4.1. 과제명
 - 4.2. 팀 정보
 - 4.3. 팀 구성원

- 5. Project Summary
 - 5.1. 문제 정의
 - 5.2. 기존 게임들과의 비교
 - 5.3. 제안 내용
 - 5.4. 기대 효과 및 의의
 - 5.5. 주요 기능 리스트
 - 5.6. 프로젝트 진척 상황(2025.03.27 기준)

- 6. Project Design
 - 6.1. 요구사항 정의
 - 6.2. 전체 시스템 구성
 - 6.3. 주요 엔진 및 기능 설계
 - 6.4. 주요 기능의 구현
 - 6.5. 기타

1. Team Information

1.1 과제명

Friendly: 플레이어의 반응에 따라 동적인 공포 이벤트를 제공하는 AI 심리 공포 게임

1.2 팀 정보

- 5. 팀 번호: 14
- 6. 팀명: BestFriend
- 7. 트랙: 산학
- 8. 지도교수: 민동보 교수

1.3 팀 구성원

- 4. 양동선(2271037): 팀 리더, 자료 제출, 기획 및 개발, AI 모델 구현, 씬 제작 및 공포 연출 구현, SFX
- 5. 박주영(2271030): 시스템 UI 및 스크립트, 에셋 관련 프로그래밍, 디자인, 아트 및 에셋 제작
- 6. 윤소진(2376179): AI 모델 설계 및 학습, 사운드 디자인, 공포 연출 및 게임 씬 제작

2. Project Summary

2.1 문제 정의

본 프로젝트는 기존 공포게임이 가지는 한계를 해결하고, 플레이어별로 다른 공포 경험을 제공하는 지능형 공포게임 시스템을 구현하는 것을 목표로 한다.

현재 다수의 공포게임은 사전에 설계된 이벤트와 연출을 반복적으로 제공하는 구조를 가진다. 이러한 구조에서는 플레이어가 여러 번 게임을 반복할수록 특정 구간의 위협 요소와 발생 타이밍을 예측하게 되며, 결과적으로 몰입감과 긴장감이 감소하는 문제가 발생한다. 즉, 공포 연출이 고정되어 있다는 점이 첫 번째 핵심 Pain point이다.

두 번째 Pain Point는 공포 반응의 개인차를 충분히 반영하지 못한다는 점이다. 공포를 유발하는 요소는 사람마다 다르게 나타난다. 어떤 플레이어는 시각적 왜곡이나 기괴한 이미지에 더 큰 공포를 느끼는 반면, 다른 플레이어는 갑작스러운 소리나 추적 상황, 점프스케어에 더 민감하게 반응할 수 있다. 그러나 기존 게임은 대부분 동일한 공포 이벤트를 모든 플레이어에게 동일하게 제공하므로, 플레이어 개개인의 특성과 반응을 반영한 맞춤형 공포 경험을 제공하기 어렵다.

따라서 본 프로젝트는 다음의 두 가지 Pain Point를 해결 대상으로 설정한다.
첫째, 반복 플레이 시 고정된 공포 연출로 인해 긴장감이 약화되는 문제이다.
둘째, 사람마다 다른 공포 반응을 기존 게임이 충분히 반영하지 못하는 문제이다.

이와 같은 문제를 해결하기 위해, 본 연구에서는 플레이어의 행동 및 반응 데이터를 수집하고 이를 분석하여 공포 이벤트의 종류, 강도, 발생 시점을 동적으로 조정하는 AI 기반 공포 연출 시스템을 제안한다.

2.2 기존 게임들과의 비교

기존 공포게임은 공포 분위기 조성, 추적 시스템, 점프스케어, 심리적 압박 등 다양한 방식으로 플레이어에게 두려움을 유발해 왔다. 그러나 대부분의 게임은 개발자가 사전에 설계한 이벤트 시나리오 안에서 공포를 제공한다는 공통적인 한계를 가진다.

우선, 다수의 인디 공포게임은 정적인 이벤트 구조를 가진다. 특정 공간에서 특정 이벤트가 발생하도록 미리 설정되어 있기 때문에, 플레이어는 반복 플레이를 통해 공포 요소의 위치와 타이밍을 쉽게 학습하게 된다. 이는 초기 플레이에서는 긴장감을 줄 수 있으나, 반복 플레이에서는 공포의 효과가 빠르게 감소한다. 예를 들어, Spooky's Jump Scare Mansion과 같은 게임은 비교적 다양한 점프스케어와 분위기 전환을 제공하지만, 본질적으로는 개발자가 설계한 공포 시나리오 중 일부가 선택되어 나타나는 구조에 가깝다. 즉, 이벤트의 조합이나 순서에는 변동이 있을 수 있으나, 플레이어 개인의 반응 데이터를 실시간으로 분석하여 연출을 조정하는 수준에는 이르지 못한다.

Evil Nun, Shadow Corridor와 같은 게임은 추적과 긴장 유지를 핵심 공포 요소로 사용한다. 이들 게임은 인공지능 기반 적 캐릭터의 이동이나 탐색 행동을 포함하고 있으나, 대체로 NPC의 추적 및 순찰 로직에 초점이 맞추어져 있다. 따라서 AI가 플레이어의 감정 반응이나 행동 패턴을 해석하여 공포 연출의 종류 자체를 동적으로 재구성하는 방식과는 차이가 있다.

최근에는 Mimesis, Phasmophobia와 같이 플레이어의 반응을 반영하여 AI를 사용해 플레이어 경험의 다양성과 예측 불가능성을 제공하는 게임들도 있지만, 이런 게임들은 주로 멀티플레이 상호작용이나 협동 구조에 의해 긴장감이 발생한다. 즉, 다른 플레이어의 행동도 변수로 작용하여, 단일 플레이어의 반응 데이터를 바탕으로 개인 맞춤형 공포를 설계하는 구조는 아니다. 이 때문에 솔로 플레이 환경에서 플레이어 개인에게 최적화된 공포 경험을 제공하는 데에는 한계가 있다.

이에 비해 본 프로젝트에서 제안하는 Friendly는 플레이어의 마우스 움직임, 음성 크기, 행동 패턴 등 다양한 반응 데이터를 바탕으로 현재 플레이어의 긴장 상태와 공포 반응을 추정하고, 이를 토대로 공포 이벤트의 종류와 발생 시점을 동적으로 선택한다는 점에서 기존 게임과 차별성을 가진다. 다시 말해, 기존 게임이 “미리 만들어 둔 공포 시나리오를 제공”하는 구조라면, Friendly는 “플레이어 반응을 기반으로 AI가 공포 연출을 선택·조정”하는 구조이다. 이를 통해 반복 플레이에서도 새로운 공포 경험을 제공하고, 플레이어별로 다른 맞춤형 공포를 구현할 수 있다.

2.3 제안 내용

본 프로젝트가 다루는 핵심 문제는 기존 공포게임이 가진 고정적이고 반복적인 공포 연출 구조, 그리고 플레이어별 공포 차이를 반영하지 못하는 한계이다. 현재 대부분의 공포게임은 특정 위치, 특정 시점, 특정 조건에서 정해진 이벤트가 매 플레이마다 동일하게 발생하도록 설계되어 있으며, 이로 인해 플레이어는 반복 플레이를 통해 공포 요소를 예측할 수 있게 된다. 또한 플레이어가 어떤 자극에 더 민감하게 반응하는지에 대한 분석 없이 동일한 공포 요소를 제공하므로, 개인에게 최적화된 공포 경험은 공포 게임 세계에서 찾기 어렵다.

이러한 문제를 해결하기 위해 본 프로젝트에서는 플레이어 반응 기반 동적 공포 연출 시스템을 제안한다. 본 시스템은 Unity 3D 환경에서 구현되는 공포게임 Friendly로 구현되며, 플레이어의 행동 및 반응 데이터를 실시간으로 수집하고, 이를 AI가 분석하여 가장 적절한 공포 이벤트를 선택·실행하는 구조로 설계된다.

구체적으로, 시스템은 먼저 플레이어의 행동 데이터와 반응 신호(Fear Signal)를 수집한다. 예를 들어 마우스 움직임의 급격한 변화는 플레이어의 긴장 상태를 나타내는 지표로 활용될 수 있으며, 음성 데시벨의 급상승은 놀람이나 공포 반응의 신호로 해석될 수 있다. 이러한 데이터를 바탕으로 시스템은 플레이어의 현재 상태를 추정하고, 어떤 유형의 공포 자극에 더 크게 반응하는지 분석한다.

그 다음 단계에서는 PPO 기반 강화학습 AI가 현재 상태와 과거 반응 데이터를 바탕으로 적절한 공포 이벤트를 선택한다. 이때 공포 이벤트는 시각적 연출, 청각적 연출, 인지 왜곡의 공포, 환경적 공포, 점프 스케어 등 다양한 범주로 구성된다. AI는 플레이어가 특정 유형의 자극에 강한 반응을 보였을 경우 해당 범주의 이벤트 가중치를 높이고, 반응이 약하거나 반복으로 인해 익숙해진 요소는 빈도를 낮추는 방식으로 공포 연출을 조정한다. 이를 통해 공포 이벤트는 고정되지 않고, 플레이어의 특성과 플레이 상황에 따라 매번 다르게 나타나게 된다.

본 프로젝트의 solution은 크게 세 가지로 정리할 수 있다.
첫째, Fear Signal 수집 및 분석을 통해 플레이어의 상태를 정량적으로 해석한다.
둘째, 강화학습 기반 AI 모델을 활용하여 공포 이벤트 선택을 자동화하고 최적화한다.

셋째, 동적 이벤트 실행 구조를 통해 동일한 맵과 동일한 공간에서도 플레이할 때마다 다른 공포 경험이 발생하도록 한다.

결과적으로 본 프로젝트는 공포게임의 반복성과 획일성을 줄이고, 플레이어의 반응에 적응하는 개인 맞춤형 공포 경험을 제공하는 것을 목표로 한다. 이는 단순히 이벤트를 무작위로 배치하는 수준을 넘어, 플레이어 데이터에 근거하여 공포의 종류와 타이밍을 조정하는 지능형 시스템이라는 점에서 기존 공포게임과 차별화된다. 따라서 Friendly는 반복 플레이에서도 예측하기 어려운 긴장감을 유지할 수 있으며, 궁극적으로는 “사람마다 다른 공포를 학습하여 제공하는 적응형 공포게임”의 가능성을 제시한다.

2.4 기대 효과 및 의의

본 프로젝트의 기대 효과는 다음과 같다.

4. 개인 맞춤형 공포 경험 제공

플레이어마다 더 크게 반응하는 공포 자극이 다르다는 점을 반영하여, 각 사용자에게 더 효과적인 공포 연출을 제공할 수 있다.

5. 반복 플레이 시 긴장감 유지

같은 공간이라도 이벤트가 매번 달라지기 때문에, 플레이어는 특정 구간을 외워서 안심할 수 없고 예측 불가능성이 유지된다.

6. 플레이어 반응이 이끄는 공포 연출

단순히 스토리와 공포를 병렬적으로 배치하는 것이 아니라, 스토리 진행 중 수집한 플레이어 반응이 이후 공포 연출 방식에 직접 영향을 미친다.

2.5 주요 기능 리스트

11. 기초 컨트롤 기능

- a. 달리기, 걷기, 방향키, 아이템 상호작용 등 플레이어가 게임 세계를 탐색하기 위한 기본적인 조작 시스템이다.

12. 오브젝트 조사 기능

- a. 게임 씬 내에서 스토리, 게임 진행에 영향을 끼치는 object와 interact하여 게임의 몰입도를 높인다. 또한 오브젝트 interacting 중 발생하는 공포 이벤트로 플레이어에게 공포 체험을 제공한다.

13. 아이템 획득 및 인벤토리 관리 기능

- a. 플레이어가 게임 내 오브젝트를 획득하면 이를 인벤토리에 저장하고, 이후 스토리 진행이나 퍼즐 해결에 활용할 수 있는 시스템이다. 획득한 아이템은 인벤토리 창에서 설명과 상태 정보 등을 확인할 수 있으며, 플레이어는 이를 통해 단서를 조합하고 오브젝트를 활용할 수 있다.

14. 키패드 등 퍼즐 상호작용 기능

- a. 획득한 아이템이나 조사한 오브젝트에서 얻은 단서들을 조합해 키패드 비밀번호나 금고 비밀번호 등을 알 수 있다. 퍼즐 요소를 통해 게임의 오락성을 높이고 플레이어에게 흥미 요소를 제공한다.

15. JSON 기반 대화 및 아이템 데이터 관리 기능

- a. json 기반 softcoding으로 콘텐츠와 코드를 분리하여 효율적으로 전체 시스템과 데이터를 관리하며, 유지보수성을 높인다.

16. 플래그/변수 기반 이벤트 분기 기능

- a. 플레이어의 행동 결과를 플래그와 변수로 저장하고, 이를 바탕으로 이후 대사, 이벤트, interacting 등의 분기점과 전개를 달라지도록 하는 시스템이다. 이를 통해 게임은 플레이어의 선택과 진행상태를 반영하는 동적 흐름을 구성한다.

17. Fear Signal 수집 및 계산 기능

- a. 플레이어의 행동 데이터를 기반으로 공포 반응 지표를 수집하고 계산하는 시스템이다. 이동 속도, 체류 시간, 급격한 마우스 움직임, 데시벨 상승 등을 종합하여 수치화한다.

18. PPO 기반 AI 공포 이벤트 선택 기능

- a. Fear Signal을 기반으로 플레이어가 공포에 무너지거나 과도하게 지치지 않도록 ML-Agents가 최적의 공포 이벤트를 선택하여 플레이 경험을 동적으로 조정한다.

19. 동적 공포 이벤트 실행 기능

- a. ML-Agents가 선택한 공포 이벤트를 실제 씬에 반영하여 공포 연출을 실행한다.

20. UI 시스템

- a. 플레이어가 게임 상태와 상호작용 정보를 확인할 수 있도록 타이틀, 인벤토리, 상호작용, 퍼즐 등 각 상황에 맞는 인터페이스를 제공한다.

~~2.6 프로젝트 진척 상황(2025.03.27 기준)~~

~~2.6.1 플레이어 이동 및 탐험 기능~~

- ~~4. Unity 6 기반의 1인칭 플레이어 이동 시스템 구현~~
- ~~5. Rigidbody 기반 이동, 점프, 마우스 시점 회전 기능 구현~~
- ~~6. 1층, 2층, 야외 등 복수의 씬을 탐험할 수 있는 구조 설계~~

~~2.6.2 씬 전환 및 시스템 유지 기능~~

- ~~5. BootstrapScene 기반 초기 로딩 구조 설계~~
- ~~6. PersistentRoot 기반 DontDestroyOnLoad 시스템 구현~~
- ~~7. PlayerSpawner를 통해 씬 진입 시 플레이어 자동 생성 기능 구현~~
- ~~8. 특정 Gameplay Scene 단독 실행 시에도 시스템이 동작할 수 있도록 Editor 전용 AutoBootstrap 구조 적용~~

~~2.6.3 상호작용 기능~~

- ~~4. Raycast 기반 오브젝트 감지 시스템 구현~~
- ~~5. 플레이어가 조사 가능한 오브젝트에 접근했을 때 상호작용 안내 텍스트 출력 기능 구현~~
- ~~6. 아이템 획득, 오브젝트 조사, 문 및 퍼즐 상호작용을 위한 기본 인터페이스 구조 설계~~

~~2.6.4 아이템 및 인벤토리 기능~~

- ~~6. ScriptableObject 기반 ItemData 구조 설계~~
- ~~7. 아이템 획득 시 인벤토리에 추가되는 기능 구현~~
- ~~8. 인벤토리 패널에서 아이템 목록을 표시하는 UI 구조 구현~~
- ~~9. 선택한 아이템의 아이콘, 이름, 설명을 상세 패널에 표시하는 기능 구현~~
- ~~10. 아이템 획득 후 UI 반영 및 데이터 저장 흐름 검증~~

~~2.6.5 UI 전환 및 입력 제어 기능~~

- ~~5. Title UI, Pause UI, Inventory UI 기본 구조 구현~~

- ~~6. Pause 시스템 구현 및 ESC 입력 처리 기능 적용~~
- ~~7. Inventory와 Pause 상태에서 Cursor lock/unlock 및 ActionMap 전환 기능 구현~~
- ~~8. Pause 시스템과 Inventory 시스템 간 입력 충돌 문제를 수정하여 UI 상태 전환 안정성 검증~~

~~2.6.6 스토리 진행 및 데이터 관리 기능~~

- ~~6. StoryBoard 기반 스토리 진행 구조 설계~~
- ~~7. JSON 기반 대화 데이터 관리 구조 설계~~
- ~~8. JSON/데이터 기반 아이템 정보 관리 구조 설계~~
- ~~9. 플래그 및 변수 조건에 따라 이벤트가 분기될 수 있는 구조 기획~~
- ~~10. 탐험, 조사, 아이템 획득, 퍼즐 해결을 통해 스토리가 단계적으로 진행되는 흐름 검증~~

~~2.6.7 공포 이벤트 설계 기능~~

- ~~3. 공포 이벤트를 시각적 자극, 청각적 자극, 점프 스케어, 인지 왜곡, 환경적 공포 등으로 분류~~
- ~~4. 각 공포 카테고리별 예시 연출 기획~~

~~2.6.8 현재까지 검증 완료한 내용~~

- ~~7. BootstrapScene 기반 초기화 정상 동작 확인~~
- ~~8. 씬 단독 실행 시 PersistentRoot 자동 생성 동작 확인~~
- ~~9. 플레이어 스폰 및 씬 진입 흐름 확인~~
- ~~10. 아이템 획득 후 인벤토리 반영 여부 확인~~
- ~~11. Pause/Inventory 전환 시 입력맵 및 커서 상태 전환 확인~~
- ~~12. 상호작용 UI 표시기능 확인~~

3. Project Design

3.1 요구사항 정의

3.1.1 기능 요구사항

11. 플레이어는 1인칭 시점으로 공간을 이동하고 탐험할 수 있어야 한다.
12. 플레이어는 오브젝트를 조사하고, 단서를 획득할 수 있어야 한다.
13. 플레이어는 아이템을 획득하고 인벤토리에서 확인할 수 있어야 한다.
14. 플레이어는 퍼즐 요소와 상호작용하여 스토리를 진행할 수 있어야 한다.
15. 대화와 아이템 정보는 JSON 기반 데이터로 관리되어야 한다.
16. 이벤트는 플래그 및 변수에 따라 분기되어야 한다.
17. 시스템은 마우스 움직임과 데시벨 데이터를 수집하여 Fear Signal을 계산해야 한다.
- ~~18. AI는 현재 플레이어 반응 상태를 바탕으로 공포 이벤트를 선택해야 한다.~~
19. 공포 이벤트는 시각적 자극, 청각적 자극, 점프스케어, 인지 왜곡, 환경적 공포 등으로 다양하게 구성되어야 한다.
20. AI는 동일한 공포 연출을 연속 3회 이상 실행하지 않는다.
21. AI는 공포 연출 후 플레이어 반응이 감소하면 다른 연출로 전환한다.
22. AI는 공포 연출 후 적절한 침묵 구간을 유지하여 긴장감을 빌드업한 뒤 다음 연출을 실행한다.
23. AI는 실시간 반응 신호와 1부에서 수집한 플레이어 성향 데이터를 결합하여 개인화된 공포 연출을 선택한다.
24. AI는 위치 기반 연출(마네킹, 조명, 문 등)을 해당 오브젝트가 없는 구역에서 실행하지 않는다.
25. 게임은 동일 구간 반복 시에도 예측 불가능한 긴장감을 유지해야 한다.

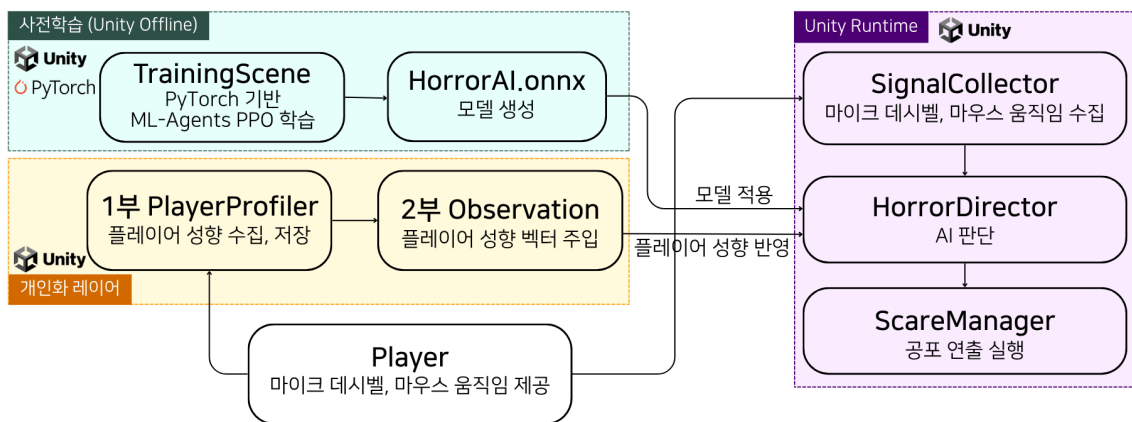
3.1.2 비기능 요구사항

6. Unity 엔진 기반에서 안정적으로 동작해야 한다.

7. Scene 전환 및 UI 상호작용이 자연스럽게 연결되어야 한다.
8. 데이터 구조는 유지보수 및 확장이 가능해야 한다.
9. 공포 이벤트 선택 시스템은 실시간 추론이 가능할 정도로 가벼워야 한다.
10. JSON 및 ScriptableObject 기반 구조를 통해 콘텐츠 추가가 용이해야 한다.

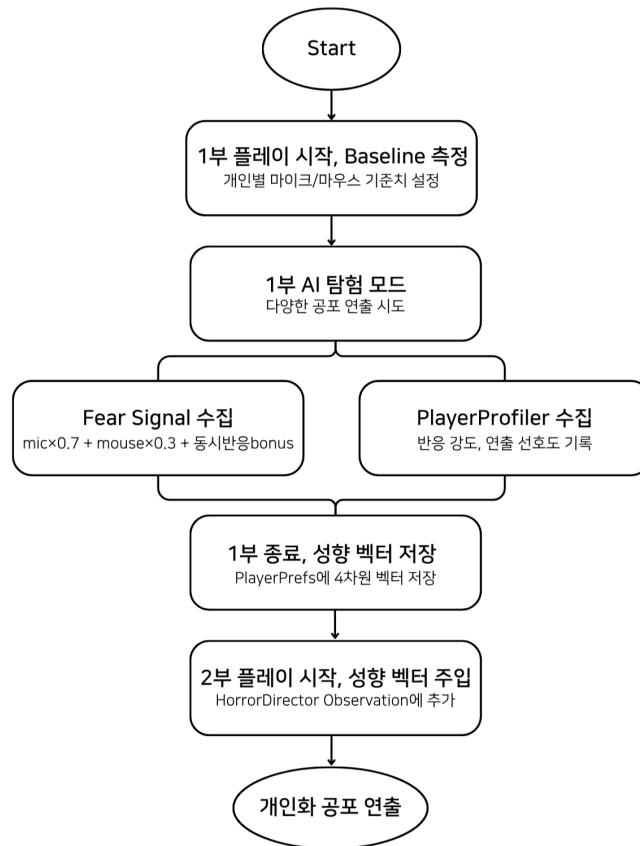
3.2 전체 시스템 구성

3.2.1 시스템 구조도 (기존 구조도와 다르게 새로 생성함)



Friendly는 Unity 기반 적응형 공포 게임으로, 사전 학습 단계와 런타임 단계로 분리된 구조이다. 사전 학습은 Unity Offline 환경에서 ML-Agents와 PyTorch를 활용해 AI 모델을 훈련하고, 런타임은 학습된 모델을 Unity에 탑재해 실시간으로 공포 연출을 제어한다. 개인화 레이어가 플레이어별 맞춤 경험을 제공한다.

3.2.2 시스템 흐름도 (기존 구조도와 다르게 새로 생성함)



3.2.3 전체 SW 구조 (파일 트리 형식)

```

Assets
├── AssetStore
├── Audio
├── Scripts
│   ├── Editor
│   └── dialogue
├── Scenes
├── ML-Agents
│   └── Config # YAML 학습 설정 파일
├── Models # 캐릭터 3D 모델
├── StreamingAssets # JSON 경로 고정 (이동 불가)
├── Settings
├── TextMesh Pro
├── QuickOutline
├── Items_Interact # 아이템 데이터 및 스프라이트
├── KeypadSafe # 키패드 관련 리소스
└── Noah_Original_Source # 캐릭터 초상화, 아트웍 등
  
```

3.2.4 전체 시스템 개요

본 시스템은 크게 게임 플레이 계층, 데이터 관리 계층, AI 공포 연출 계층, UI 계층으로 구성된다. 각 SW 계층이 수행하는 역할은 다음과 같다.

1. 게임 플레이 계층

- a. Unity 6 기반 1인칭 플레이어 관리
- b. 플레이어의 이동, 시점 회전, 점프, 상호작용 관리
- c. 플레이어의 씬 탐험(1층, 2층, 야외 등) 관리
- d. 퍼즐 요소 및 아이템 연동 관리

2. 데이터 관리 계층

- a. JSON 기반 대화 데이터 관리
- b. JSON/ScriptableObject 기반 아이템 데이터 관리
- c. 플래그/변수 기반 스토리 분기 관리
- d. StoryBoard와 시나리오를 코드와 연결하는 softcoding 구조 관리

3. AI 공포 연출 계층

- a. Player 행동 데이터 수집
- b. Fear Signal 계산
- c. PPO 기반 AI 강화학습 수행
- d. 최적 공포 이벤트 선택
- e. 이벤트 실행 후 반응 분석

4. UI 계층

- a. 타이틀 UI 관리
- b. 인벤토리 UI 관리
- c. 상호작용 UI 관리
- d. 퍼즐 UI 관리
- e. 스토리 진행에 맞는 HUD 및 연출 UI 관리

3.3 주요 엔진 및 기능

3.3.1 Unity 엔진 기반 게임 프레임 워크 사용

전체 게임 시스템은 Unity 6 (6000.0.62f1) 를 기반으로 구현하였다. Unity 엔진은 씬 단위 레벨 관리, 물리 처리, UI 구성, 애니메이션, 오디오, 스크립트 제어를 통합적으로 지원하므로, 탐험형 공포 게임의 구조를 구현하기에 적합하다고 판단했다. 특히 본 프로젝트에서는 BootstrapScene + PersistentRoot + Gameplay Scene 구조를 도입하여, 초기 시스템 로딩과 실제 플레이 씬을 분리하고, UI 및 핵심 매니저를 DontDestroyOnLoad 방식으로 유지하도록 설계하였다. 이를 통해 씬 전환 시에도 플레이어의 정보는 계속 연결되며, 인벤토리, 일시정지, 상호작용 UI 등 주요 시스템이 안정적으로 동작하도록 하였다.

3.3.2 Player 시스템 및 입력 처리

전체 플레이어 조작은 Rigidbody 기반 1인칭 플레이어 컨트롤러로 구현하였다. 이동, 점프, 시점 회전, 상호작용 기능을 하나의 PlayerController 모듈에서 관리하며, 충돌 및 중력 처리는 Unity 물리 시스템을 활용하였다.

입력 처리에는 Unity6의 최신 기능인 Unity New Input System을 도입하였다. 입력 액션은 Player와 UI Action Map으로 분리하여, 일반 플레이 상태와 인벤토리/퍼즐/UI 조작 상태를 명확히 구분하였다. 이를 통해 게임 플레이 중에는 이동 및 시점 조작이 가능하고, 인벤토리나 퍼즐 조작 시에는 UI 입력으로 안전하게 전환되도록 설계하였다.

3.3.3 인게임 상호작용 및 퍼즐 시스템

전반적인 상호작용 시스템은 Raycast 기반 감지 방식으로 구현하였다. 플레이어의 카메라 전방으로 Raycast를 발사하여 상호작용 가능한 오브젝트를 탐지하고, IInteractable 인터페이스를 구현한 오브젝트에 대해 공통 상호작용 로직을 적용하였다. 이를 통해 인터페이스가 구현된 오브젝트에는 ‘Hold [Z] to interact.’(데모용 문구) 라는 상호작용 문구가 표시되며, 아이템의 테두리에 흰색 outline이 표시된다. 이 구조를 통해 아이템 획득, 문 열기/닫기, 퍼즐 인터랙션,

조사형 오브젝트 감지 등을 하나의 일관된 방식으로 처리할 수 있도록 하였다.

퍼즐 시스템은 키패드, 문, 조사형 오브젝트 등을 독립 모듈로 분리하여, 상호작용 결과가 플래그/변수 기반 스토리 진행과 연동되도록 설계하였다.

3.3.4 인벤토리 및 UI 시스템

인벤토리 시스템은 ScriptableObject 기반 ItemData 구조와 이벤트 기반 UI 갱신 방식으로 구현하였다. 각 아이템의 정보를 담을 수 있는 구조가 필요하다고 판단했고, 이에 ItemData라는 새로운 데이터 구조를 구현했다. itemID, item_name, description, image를 저장하는 구조이다.

아이템 획득 시 ItemData(itemID, item_name, description, image)를 리스트에 저장하고, OnInventoryChanged 이벤트를 통해 UI가 자동으로 갱신되도록 구성하였다. UI가 갱신되면 인벤토리에 획득한 아이템이 정보와 함께 나타나게 된다.

UI는 UICanvas 하위에 상호작용 텍스트, 인벤토리 패널, 퍼즐 패널, Pause 패널 등을 계층적으로 배치하였다. 이를 통해 UI 상태별 표시/비표시 전환과 입력 전환을 명확히 처리할 수 있도록 하였다.

3.3.5 데이터 관리 구조

스토리, 대화, 아이템 정보, 이벤트 분기 데이터는 JSON + ScriptableObject 혼합 구조로 관리하였다.

대화 및 스토리 진행 정보는 JSON을 통해 softcoding 형태로 관리하여, 코드 수정 없이도 대사나 분기 조건을 변경할 수 있게 하였다.

아이템 정보는 ScriptableObject를 통해 에디터 상에서 직관적으로 관리하고, 아이콘, 이름, 설명 등 UI 표시 데이터와 쉽게 연결되도록 하였다.

또한 플래그 및 변수 기반 분기 구조를 도입하여, 특정 아이템 획득 여부나 퍼즐 해결 여부에 따라 다음 이벤트가 달라지는 구조를 구현하였다.

3.3.6 AI 공포 연출 기능

본 프로젝트의 핵심 기능은 플레이어의 반응 데이터를 기반으로 공포 이벤트를 동적으로 선택하는 AI 공포 연출 시스템이다.

Fear Signal은 마우스 움직임 변화량과 마이크 데시벨 변화량을 기반으로 계산하며, 이를 통해 플레이어의 놀람·긴장 반응을 간접적으로 추정한다.

AI 모델은 Unity ML-Agents 기반 PPO 정책 학습을 통해 개발 단계에서 사전 학습하고, 실제 게임 실행 시에는 추론 형태로 공포 이벤트를 선택하도록 적용할 계획이다.

이벤트 종류는 시각적 자극, 청각적 자극, 점프스케어, 인지 왜곡, 환경 변화 등으로 구분하여, 같은 맵에서도 플레이어마다 다른 공포 경험을 제공하도록 설계하였다.

3.4 주요 기능의 구현

3.4.1 플레이어 이동 및 상호작용 기능의 구현

(1) PlayerController 모듈 구현

플레이어 이동 및 시점 제어를 위해 PlayerController 모듈을 구현하였다.

본 모듈은 (a)이동 입력 처리, (b)시점 회전 처리, (c)점프 및 달리기 처리, (d)상호작용 오브젝트 감지 및 실행 기능을 수행한다.

이 중 (a)이동 입력은 Unity New Input System으로부터 전달된 Vector2 값을 읽어, 플레이어의 전진/후진 및 좌우 이동 방향으로 변환하는 방식으로 구현하였다. 이동은 Rigidbody의 속도 벡터를 제어하는 방식으로 처리하였다.

(b)시점 회전은 마우스 입력값을 누적하여 카메라 pitch와 플레이어 yaw를 분리 제어하는 방식으로 구현하였다.

(c)점프는 바닥 충돌 여부를 확인한 뒤 Rigidbody에 upward impulse를 적용하는 방식으로 구현하였다.

(2) 상호작용 감지 알고리즘

A4 상호작용 기능은 플레이어 카메라 전방으로 Raycast를 발사하여 구현하였다. 처리 알고리즘은 다음과 같다.

1. 카메라 위치와 전방 방향을 기준으로 Ray를 생성한다.
2. 지정한 상호작용 거리 내에서 Raycast를 수행한다.
3. 충돌한 collider에서 IInteractable 컴포넌트를 검색한다.
4. 자식 collider를 맞는 경우도 처리하기 위해
GetComponentInParent<IInteractable>()까지 탐색한다.
5. 상호작용 가능한 오브젝트가 존재하면 상호작용 안내 UI를 활성화하고,
입력 시 Interact()를 호출한다.

이 구조를 통해 문, 아이템, 조사형 오브젝트, 퍼즐 오브젝트를 동일한 방식으로 처리할 수 있도록 하였다.

3.4.2 인벤토리 시스템의 구현

(1) ItemData 및 InventoryManager 모듈 도입

인벤토리 기능 구현을 위해 ItemData ScriptableObject와 InventoryManager 모듈을 도입하였다.

- B1. 아이템 정보 관리
- B2. 아이템 획득 처리
- B3. 인벤토리 리스트 저장
- B4. 인벤토리 UI 동기화

ItemData는 아이템 이름, 설명, 아이콘, 분류 정보를 저장하며, 에디터 상에서 각 아이템을 개별 asset으로 관리할 수 있도록 하였다.

InventoryManager는 획득한 아이템 리스트를 관리하며, 아이템이 추가될 때마다 OnInventoryChanged 이벤트를 발생시켜 UI를 갱신한다.

(2) 아이템 획득 처리 방식

아이템 획득 기능은 ItemPickup 모듈에서 구현하였다.

플레이어가 상호작용 키를 입력하면, 해당 오브젝트의 Interact()가 호출되고, 연결된 ItemData를 InventoryManager.AddItem()에 전달한다.

이후 오브젝트는 씬에서 제거되며, 인벤토리 UI는 이벤트를 통해 자동으로 갱신된다.

(3) 인벤토리 UI 구현

인벤토리 패널은 좌측 아이템 목록과 우측 상세 정보 패널로 구성하였다.

좌측 목록에는 아이템 이름이 동적으로 생성되며, 특정 아이템 선택 시 우측 패널에 아이콘, 이름, 설명이 표시되도록 구현하였다.

또한 인벤토리를 열 때는 게임 시간을 정지하고 커서를 unlock하여 UI 조작이 가능하도록 하였으며, 닫을 때는 다시 플레이 입력 상태로 복귀하도록 설계하였다.

3.4.3 퍼즐 및 문 상호작용 기능의 구현

(1) 퍼즐 기능 구현

퍼즐 기능은 스토리 진행에 필요한 플래그 기반 상태 전환을 중심으로 구현하였다.

예를 들어 키패드 퍼즐은 입력값 검증 모듈과 성공/실패 분기 모듈로 나누어 구현하였다.

정답 입력 시 특정 플래그를 true로 변경하고, 해당 플래그를 참조하는 다른 문 또는 이벤트가 해제되도록 설계하였다.

(2) 문 열기/닫기 기능 구현

문 시스템은 DoorInteractable과 DoorAutoCloseTrigger 두 모듈로 구성하였다.

- DoorInteractable : 플레이어 상호작용으로 문을 열고 닫는 기능
- DoorAutoCloseTrigger : 플레이어가 문 통로를 지나간 뒤 자동으로 문을 닫는 기능

문은 루트 오브젝트와 회전축(HingePivot)을 분리하여 구현하였다.

문을 막는 실제 collider는 회전축 하위에 배치하여, 문이 열릴 때 collider도 함께 회전하도록 하였다. 문이 플레이어 방향으로 들어받지 않도록, 문과 플레이어 위치 관계를 기반으로 열리는 방향을 결정하였다. 자동 닫힘은 문 통로를 덮는 trigger collider에서 플레이어가 빠져나가는 시점(OnTriggerExit)을 감지하여 수행하였다. 이 과정을 통해 문이 열리는 도중 즉시 닫히는 문제를 줄이고, 실제 플레이어 통과 이후 자연스럽게 닫히도록 하였다.

3.4.4 Fear Signal 수집 및 계산 기능의 구현

(1) 반응 데이터 수집 모듈

Fear Signal 계산을 위해 두 가지 입력 신호를 수집하였다.

- C1. 마우스 움직임 변화량
- C2. 마이크 데시벨 변화량

C1은 프레임 간 마우스 이동량의 크기를 Vector2 magnitude로 계산한다. C2는 Unity Microphone API를 사용하여 입력된 오디오 샘플의 최대 진폭을 측정하고 0~1 사이의 정규화된 값으로 변환하여 사용한다. 두 신호 모두 게임 성능 부하를 줄이기 위해 매 프레임이 아닌 0.1초 간격으로 수집한다.

(2) 전처리 방식

Fear Signal은 단순 절대값이 아니라 개인별 Baseline 대비 변화량을 기준으로 계산한다. 게임 시작 후 10초간 플레이어의 평균 마우스 변화량과 평균 마이크 데시벨을 측정하여 개인별 Baseline으로 설정한다. 이후 수집되는 신호는 Baseline을 차감한 초과분만을 공포 반응으로 인정함으로써 개인차를 보정하고, 단순 조작 실수나 외부 소음 같은 단발성 노이즈가 직접 공포 반응으로 해석되지 않도록 하였다.

(3) 공포 반응 점수화

최종 Fear Signal은 다음 요소를 종합하여 산출한다.

$$\text{FearSignal} = (\text{정규화된 마이크} \times 0.7) + (\text{정규화된 마우스} \times 0.3) + (\text{마이크} \times \text{마우스} \times 0.3)$$

마이크(비명)에 더 높은 가중치(0.7)를 부여하고, 마우스(놀람)에 낮은 가중치(0.3)를 부여하였다. 마지막 항은 마이크와 마우스가 동시에 크게 반응할 때만 커지는 동시 반응 보너스(simultaneousBonus)로, 두 신호가 함께 반응할 경우 실제 공포 반응일 가능성이 높다는 점을 반영하였다. 이를 통해 단일 수치가 아니라 복합적인 반응 패턴을 공포 반응 점수로 해석하는 구조로 구현하였다.

3.4.5 AI 공포 이벤트 선택 기능의 구현

(1) AI 이벤트 선택 구조

AI 공포 이벤트 선택 기능은 플레이어의 현재 반응 상태를 기반으로, 어떤 유형의 공포 이벤트를 실행할지 결정하는 모듈이고, Unity ML-Agents의 PPO(Proximal Policy Optimization) 알고리즘을 기반으로 구현하였다. HorrorDirector는 매 Decision Period마다 27개의 Observation을 입력받아 8가지 action 중 하나를 선택한다.

Observation 구성은 다음과 같다.

- 현재 마이크/마우스 반응값 및 직전 Fear Signal (3개)
- 직전 행동 원-핫 인코딩 (8개)
- 연속 침묵 길이 (1개)
- 각 연출 사용 빈도 (7개)
- 위치 기반 연출 가능 여부 (4개) - (마네킹/조명/문/그림글리치)
- 플레이어 성향 벡터 (4개)

Action은 다음 8가지로 정의하였다.

0: 아무것도 안 함 (침묵)

1: 마네킹의 움직임

2: 조명 깜빡임

- 3: 점프스케어 (2D, 3D 오브젝트)
- 4: 소리 재생 (발소리/웃음소리/비명 등)
- 5: 문 열림/닫힘
- 6: 그림 클리치
- 7: 예비 슬롯 (추후 공포연출 추가 시 예비 슬롯에 추가)

위치 기반 연출(action 1, 2, 5, 6)은 해당 오브젝트가 플레이어 주변에 존재할 때만 실행되며, 불가능한 연출 선택 시 페널티를 부여하여 AI가 스스로 가능한 연출만 선택하도록 학습시켰다.

(2) 보상 설계 개요

보상함수는 AI가 특정 패턴만 반복하기보다, 플레이어 반응에 따라 더 효과적인 공포 이벤트의 종류와 타이밍을 학습하도록 다음과 같은 4개의 레이어로 구성하였다.

Fear Signal Reward: 공포 연출 실행 후 측정된 Fear Signal에 비례하여 보상을 부여한다. 연출 중 쿨타임(isActing)이 활성화된 상태에서 또 다른 연출을 시도하면 낭비 행동으로 간주하여 페널티를 부여한다.

Repetition Penalty: 동일한 연출을 연속으로 선택하거나, 같은 연출을 반복했음에도 Fear Signal이 이전보다 낮아진 경우 페널티를 부여한다. 이를 통해 AI가 특정 연출만 반복하는 것을 방지하였다.

Diversity Reward: 최근 8개 행동 이력의 정규화된 엔트로피를 계산하여, 다양한 연출을 골고루 사용할수록 보상을 부여한다.

Pacing Reward: Action 0(침묵)을 선택하여 일정 스텝 이상 침묵을 유지하면 보상을 부여한다. 단, 지나치게 긴 침묵은 방치로 간주하여 페널티를 부여한다. 이를 통해 AI가 공포 연출의 완급 조절을 스스로 학습하도록 하였다.

(3) 학습 환경 구성

실제 플레이어 없이 학습할 수 있도록 TrainingScene을 별도로 구성하였다. TrainingScene에서는 TrainingSignalSimulator가 SignalCollector를 대체하여 가상의 마이크/마우스 반응을 생성하고, ScareManager_Training이 실제 연출 없이 쿨타임 처리만 담당한다. TrainingManager는 겁쟁이/침묵형/일반/강심장 4가지 플레이어 시나리오를 순환하여 다양한 플레이어 유형에 범용적으로 대응하는 전략을 학습시켰다.

(4) 실제 적용 방식

실제 게임 실행 시에는 개발 단계에서 ML-Agents(PyTorch 기반)로 사전 학습한 PPO 정책을 ONNX 포맷으로 변환하여 Unity Inference Engine으로 추론을 수행한다. 런타임에서 추가 학습은 수행하지 않는다.

개인화는 1부와 2부로 나누어 구현하였다. 1부 플레이 동안 PlayerProfiler가 플레이어의 반응 데이터를 수집하여 4차원 성향 벡터로 압축한다.

플레이어 성향 벡터 구성:

- [0] 평균 마이크 반응 강도
- [1] 평균 마우스 반응 강도
- [2] 가장 반응이 좋았던 연출 ID (정규화)
- [3] 반응 감쇠율 (무더지는 속도)

이 벡터는 PlayerPrefs에 저장되어 썬 전환 후에도 유지된다. 2부 시작 시 해당 벡터를 불리와 HorrorDirector의 Observation에 추가 입력함으로써, 사전 학습된 범용 모델이 플레이어 개인의 반응 패턴을 반영한 개인화된 공포 연출을 수행하도록 하였다. 이는 온라인 파인튜닝이 아닌 Observation 확장을 통한 조건부 추론 방식으로, 단일 세션의 제한된 데이터만으로도 안정적인 개인화가 가능하도록 설계하였다.

3.5 기타

3.5.1 구현 과정에서의 주요 트러블 슈팅 과정

본 프로젝트는 BootstrapScene, PersistentRoot, DDOL 구조를 활용하였기 때문에, 씬 전환 시 플레이어 재스폰과 UI/입력 매니저 간 참조 재바인딩 문제가 주요 구현 이슈로 나타났다. 이를 해결하기 위해 플레이어 입력과 UI 상태 전환 책임을 분리하고, 실제 입력을 받는 PlayerController가 자신의 PlayerInput을 기준으로 ActionMap을 전환하도록 구조를 정리하였다.

또한 상호작용 UI와 문 상호작용 기능 구현 과정에서 자식 collider를 맞는 경우까지 고려하여 GetComponentInParent<IInteractable>() 방식으로 상호작용 탐색 범위를 확장하였다.

3.5.2 성능 및 실시간성 고려

공포 이벤트 선택 AI는 실시간 플레이를 방해하지 않아야 하므로, 실제 게임 내 적용 시에는 매 프레임 추론이 아니라 일정 간격 단위의 lightweight inference 방식으로 운영할 계획이다.

또한 Fear Signal 계산 역시 raw 데이터 전체를 그대로 처리하기보다, 짧은 이벤트 윈도우 기준으로 요약된 지표를 사용하는 구조로 경량화할 예정이다.

3.5.3 한계 및 향후 개선 방향

현재 Fear Signal은 마우스 움직임과 데시벨이라는 행동 기반 프록시 지표를 사용하므로, 실제 공포 감정을 완전하게 측정하는 데에는 한계가 있다.

따라서 향후에는 플레이 테스트 설문, 이벤트 직후 self-report, 추가 행동 로그 등을 함께 활용하여 지표의 타당성을 검증하고 정교화할 필요가 있다.

또한 공포 이벤트의 다양성과 반복 억제를 위해 동일 이벤트 반복 패널티, 강도 상한선, 이벤트 간 최소 간격 제어 등 추가 제어 로직을 확장할 계획이다.

3.5.4 기대 효과

본 시스템은 플레이어 반응 데이터를 게임 연출에 반영함으로써, 기존 공포 게임의 고정된 패턴에서 벗어나 보다 높은 몰입감과 반복 플레이 가치를 제공할 수 있다.

또한 Unity 기반 탐험형 퍼즐 게임과 AI 공포 연출 시스템의 결합 사례로서, 향후 감정 반응 기반 게임 연출 연구 및 인터랙티브 콘텐츠 설계에 활용 가능한 구조를 제시한다.